

Tarea 2

1. Enunciado

Desarrollar un programa utilizando el lenguaje de programación **C o C++** que implemente un **juego** que **simule** una **carrera de caballos**, utilizando la biblioteca de hebras de control **pthread.h** y de manejo de pantalla **ncurses.h**. Este programa debe tener, al menos, las funciones, métodos y/o módulos que permitan hacer uso de los siguientes procedimientos:

- Preparar_carrera
- Caballo
- Carrera
- Presentar_resultado

El avance del caballo debe ser aleatorio en 1 posición.

Los caballos pueden correr una carrera de 1, 2, 3 o 4 vueltas en una pista que puede ser de 30, 40, 50, o 60 metros.

La carrrrera puede tener entre 2 a 7 caballos en competencia.

La interfaz de salida debe mostrar las vueltas y metros recorridos por cada caballo como la sumatoria total de vueltas y metros de todos los caballos.

Una vez que todos los caballos crucen la meta, se debe entregar las características de la carrera y el resultado de la misma.

2. Programa

El código debe ser desarrollado usando una metodología de **programación modular**, es decir, debe haber uso de funciones y/o métodos que implementen de forma genérica las principales funcionalidades, las cuales serán usadas en el programa principal.

La idea de esta implementación es poder crear funciones, métodos y/o módulos que puedan ser reutilizados en cualquier otro programa que se pueda implementar en el futuro con el correspondiente ahorro en tiempo y líneas de código.

El programa debe ser desarrollado para que se ejecute sobre una plataforma Linux utilizando el compilador **gcc o g++**. Para sistema operativo Windows se recomienda el uso de **WSL – Windows Subsystem for Linux**.

El objetivo de esta implementación es poder desarrollar una o varias funciones que se ejecuten varias veces pero en distintas hebras y con distintos parametros. Esta tarea debe implementar una función de probabilidad para el avance de los caballos.

Se debe realizar el manejo de pantalla en modalidad texto utilizando la biblioteca `ncurses.h`. La siguiente figura muestra un bosquejo o layout de salida para la aplicación.

```

===== CARRERA =====

Caballo A: 0 vuelta(s) - 30 metros
Caballo B: 0 vuelta(s) - 37 metros
Caballo C: 1 vuelta(s) - 69 metros
Caballo D: 1 vuelta(s) - 41 metros
Caballo E: 2 vuelta(s) - 81 metros
Caballo F: 0 vuelta(s) - 20 metros
Caballo G: 1 vuelta(s) - 55 metros

Totales: 5 vueltas - 333 metros

Carril 1 | A | ----- | N
Carril 2 | B | ----- | N
Carril 3 | C | ----- | N
Carril 4 | D | ----- | N
Carril 5 | E | ----- | N
Carril 6 | F | ----- | N
Carril 7 | G | ----- | N
          0          DISTANCIA

Carrera N°: 32
Largo de la pista: 40 metros
Número de vueltas: 3

El caballo ganador fue: ¿?
Gracias por participar.
¿Desea configurar una nueva carrera?

```

3. Evaluación

El trabajo debe ser **original** y **no copiado desde Internet** (copiar y pegar), de lo contrario será evaluado con la nota **NCR**.

El programa que se entregue deberá ser revisado con el profesor para verificar su apropiada codificación y ejecución.

La pauta de evaluación es la siguiente:

- Avance (*trabajo efectivo desarrollado en la semana*) 20 %.
- Interfaz (*presentación por pantalla, manejo de errores, manual de instrucciones*) 10%
- Código (*originalidad, uso de funciones y estructuras de datos*) 30 %.
- Funcionalidad (*nivel de implementación de los requerimientos solicitados*) 40 %.

4. Instrucciones de Entrega

- Reunirse en los mismos grupos de la Tarea 1.
- **Solo un integrante** debe **enviar** el trabajo. **Quien envíe** debe **mencionar** a sus **compañeros**.
- Respetar las buenas prácticas de programación.
- **Entrega hasta** lo indicado **mediante** la plataforma **EVA**. Entregas fuera de plazo tendrán **descuento** de puntaje.
- **Deben adjuntar su bitácora** al trabajo entregado.